

Lab: Asymmetric Encryption Using RSA



Skills
Network

Estimated Time Needed: 30 minutes

Overview:

In this lab, you will learn how to encrypt and decrypt files using RSA encryption with OpenSSL. This will involve generating RSA keys, encrypting a file using the public key, and decrypting it using the private key.

Learning Objectives:

After completing this lab, you will be able to:

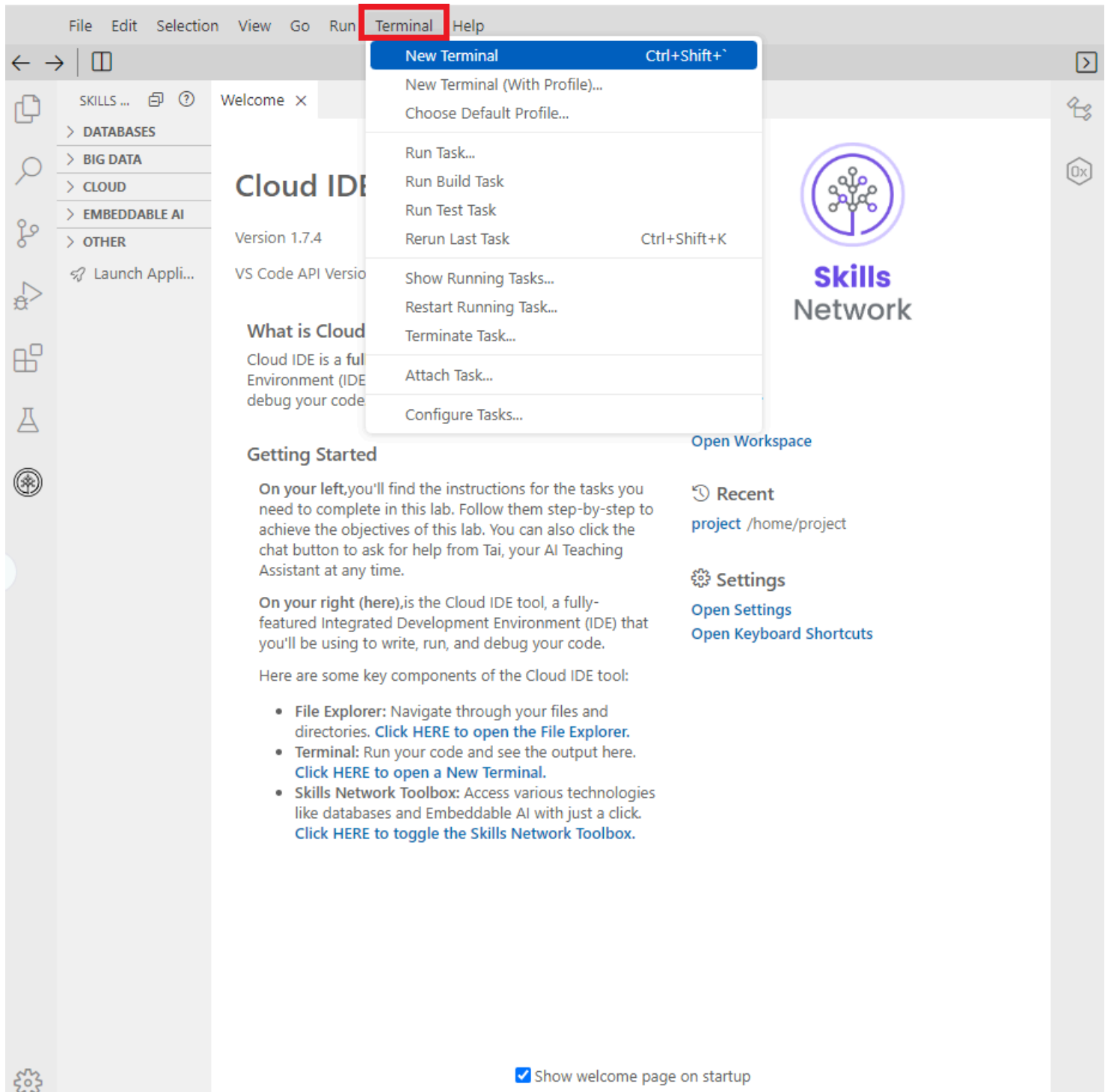
- Create RSA key pairs for encryption
- Encrypt and decrypt files using RSA

Prerequisites (Optional):

Familiarity with using the Linux command prompt.

Initializing the Lab Environment

Click Terminal to Open New Terminal



Click on New Terminal

File Edit Selection View Go Run Terminal Help

SKILLS ... Welcome x

> DATABASES

> BIG DATA

> CLOUD

> EMBEDDABLE AI

> OTHER

Launch Appli...

Cloud IDE

Version 1.7.4

VS Code API Version: 1.88.1

What is Cloud IDE?

Cloud IDE is a **fully-featured** Integrated Development Environment (IDE) that you'll be using to write, run, and debug your code.

Getting Started

On your left, you'll find the instructions for the tasks you need to complete in this lab. Follow them step-by-step to achieve the objectives of this lab. You can also click the chat button to ask for help from Tai, your AI Teaching Assistant at any time.

On your right (here), is the Cloud IDE tool, a fully-featured Integrated Development Environment (IDE) that you'll be using to write, run, and debug your code.

Here are some key components of the Cloud IDE tool:

- File Explorer: Navigate through your files and

Skills Network

Start

- New File...
- Open
- Open Workspace

Recent

- project /home/project

Settings

- Open Settings
- Open Keyboard Shortcuts

```
theia@theia-manishkr: /home/project $
```

Copy and paste the commands into the terminal for the remaining steps.

Step 1: Generate RSA Private Key

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
```

Command Description

Command/Option	Description
openssl genpkey	Generates a private key.
-algorithm RSA	Specifies the RSA algorithm for key generation.
-out private_key.pem	Output file where the private key will be stored.
-pkeyopt rsa_keygen_bits:2048	Option to specify the key size, here 2048 bits for RSA key generation.

Step 2: Extract Public Key from Private Key

```
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

Command Description

Command/Option	Description
openssl rsa	RSA-specific utility in OpenSSL, used for key management.
-pubout	Tells OpenSSL to extract the public key from the private key.
-in private_key.pem	Input file, in this case, the private key.
-out public_key.pem	Output file where the public key will be saved.

Step 3: Create a Test File

```
echo "This is a test file for RSA encryption." > test_file.txt
```

Command Description

Command/Option	Description
echo	Command to output the specified string to a file or terminal.

Command/Option	Description
“This is a test file...”	The actual text content that will be written to the file.
> test_file.txt	Redirects the output of echo to a file named test_file.txt.

Step 4: Encrypt the Test File using RSA public key

Step A: Encrypt the File

```
openssl pkeyutl -encrypt -in test_file.txt -pubin -inkey public_key.pem -out test_file_encrypted.bin
```

Command Description

Command/Option	Description
openssl pkeyutl	Utility for performing public key cryptographic operations (encryption, decryption, and so on).
-encrypt	Specifies that the operation is encryption.
-in test_file.txt	Input file, in this case, the Test File to encrypt.
-pubin	Indicates that the provided key is a public key.
-inkey public_key.pem	Specifies the public key file to use for encryption.
-out test_file_encrypted.bin	Output file where the encrypted file will be stored.

Step B: Open and Verify that File is Encrypted

```
cat test_file_encrypted.bin
```

Command/Option	Description
cat	Displays the contents of a file.
test_file_encrypted.bin	The file with the encrypted binary data.

You should see unreadable binary data, confirming that the file has been encrypted.

Step 5: Decrypt the Test File using RSA private key

Step A: Decrypt the File

```
openssl pkeyutl -decrypt -in test_file_encrypted.bin -inkey private_key.pem -out test_file_decrypted.bin
```

Command Description

Command/Option	Description
openssl pkeyutl	Utility for performing public key cryptographic operations.
-decrypt	Specifies that the operation is decryption.
-in test_file_encrypted.bin	Input file, in this case, the encrypted test file.
-inkey private_key.pem	Specifies the RSA private key to use for decryption.
-out test_file_decrypted.bin	Output file where the decrypted file will be saved.

Step B: Open and Verify that File is Decrypted

```
cat test_file_decrypted.bin
```

Command/Option	Description
cat	Displays the contents of a file.
test_file_decrypted.bin	The file with the decrypted data.

You should see the original text: **This is a test file for RSA encryption.**

Exercises

Exercise 1: Encrypt and Decrypt a Small Message Using RSA

Objective: Learn to encrypt and decrypt a short message using RSA.

Task Details:

Step 1: Generate RSA Key Pair:

► Hint!

Step 2: Create a Short Message

► Hint!

Step 3: Encrypt the Message

► Hint!

Step 4: Decrypt the Encrypted Message:

► Hint!

Exercise 2: RSA Key Pair Generation and File Encryption

Objective: Generate an RSA key pair and use it to encrypt and decrypt a file.

Task Details:

Step 1: Generate RSA Key Pair:

► Hint!

Step 2: Create a New File

► Hint!

Step 3: Encrypt the Message

► Hint!

Step 4: Decrypt the Encrypted Message:

► Hint!

Conclusion

In this lab, you successfully learned how to encrypt and decrypt files using RSA encryption with OpenSSL. By generating an RSA key pair, you were able to securely encrypt a file with the public key and decrypt it using the private key. This process demonstrated the fundamental principles of asymmetric encryption, where the encryption and decryption keys are different, ensuring secure data transmission. You can now apply these techniques to protect sensitive files and practice encryption in various real-world scenarios.

Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.